

4] Variable length positional arguments (*args)

1) Sometimes it is not fixed that how many positional arguments to pass, at that time we use variable length positional arguments

Positional argument

```
def greet (name, age):  
    print ("my name is", name,  
          "and age is", age)  
greet ("sahul", 25)
```

2 para

① ② 2 arg

2) we receive these arguments in *args.

3) It takes *args as a tuple
← Parameters

```
def calculate_sum(*args):  
    print(args)  
    print(type(args))
```

```
result = calculate_sum(1, 2, 3)  
print(result)
```

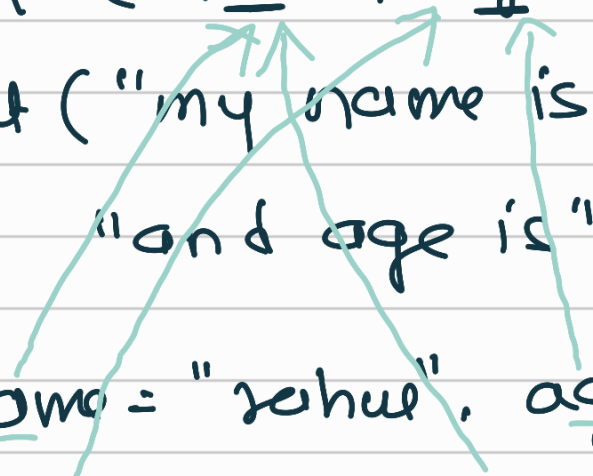
5] Variable length keyword
Arguments (**kwargs)

Keyword arguments:

```
def greet(name, age):  
    print("my name is", name,  
          "and age is", age)
```

```
greet(name = "rahul", age = 20)
```

```
greet(age = 20, name = "rahul")
```

A diagram with three green arrows. One arrow starts from the 'name' parameter in the function definition and points to the 'name' argument in the first function call. A second arrow starts from the 'age' parameter in the function definition and points to the 'age' argument in the first function call. A third arrow starts from the 'age' parameter in the function definition and points to the 'name' argument in the second function call, illustrating how keyword arguments are matched to parameters regardless of their order.

1) Using variable length keyword arguments, we can pass many keyword arguments to the function.

2) `**kwargs` can take as many keyword arguments.

3) Inside function, `kwargs` act as a dictionary

The keyword arguments we pass, we take as a key-value pair

```
def print_details (**kwargs):
```

```
    print(kwargs)
```

```
    print(type(kwargs)) → dict
```

```
    for k,v in kwargs.items():
```

```
        print(k,v)
```

```
print_details (name = "sahil", age = 25,  
               city = "pune")
```

↓
df - { "name": "zahul",
"age": 25,
"city": "pure" }